Data Science

Class Imbalance & Hypothesis Testing

Prof. Rayson Laroca
rayson@ppgia.pucpr.br

Key dates

- May 20 (today): Class Imbalance & Hypothesis Testing
- May 27: Exam #2 (20:00)
- June 3: Exam #2 feedback + project submission deadline
  - No new topics will be introduced
  - Use this time to make progress on your project
- June 10: Project Presentations (Part 1)
- June 17: Project Presentations (Part 2)
- June 24: Partial Recovery (written exam, if needed) 🇧🇷
- June 30: Extended Recovery (written exam, if needed)

Today’s goals

- Recognize common problems caused by **imbalanced data**
  - Explore practical strategies to reduce imbalance during training

- Understand the role of **statistical testing** in data-driven decision-making
  - Learn how to choose statistical tests based on the experimental setting

CLASS IMBALANCE

Class imbalance

- Class imbalance is pervasive → it occurs in most ML problems
- Occurs when the distribution of classes is highly skewed
  - High accuracy can hide poor performance on rare classes
- Examples:
  - **Credit risk**: creditworthy vs. non-creditworthy customers
  - **Fraud detection**: legitimate vs. fraudulent transactions
  - **Disease diagnosis**: non-cancerous vs. cancerous cases

The Accuracy Trap (Recap)

- A model can achieve high accuracy by mostly predicting the majority class
  - Example: if 99% of samples are negative, always predicting “negative” gives 99% accuracy

- For imbalanced data, accuracy alone is often misleading
  - Prefer metrics such as precision, recall, F1-score, balanced accuracy, ROC-AUC, and PR-AUC

Handling Class Imbalance

- Many ML algorithms assume a balanced class distribution (e.g., 50/50%, 33/33/33%, etc.)

- Two common data-level strategies for handling class imbalance:
  - **Oversampling:** increase the number of minority-class samples
  - **Undersampling:** reduce the number of majority-class samples

  - These strategies modify the **training data distribution**
    - They are widely used because they are simple, model-independent, and easy to combine with different algorithms

Handling Class Imbalance – Extra

- Other strategies also exist, such as:
  - **Class weighting**: assigns greater importance to underrepresented classes during training, making errors in these classes more costly
    - This does not change the number of samples in the dataset
  - **Threshold tuning**
  - **Cost-sensitive learning**
  - **Ensemble-based methods**

OVERSAMPLING

Oversampling

- Increases the number of minority-class samples
  - Rationale: help the model see more minority-class examples during training

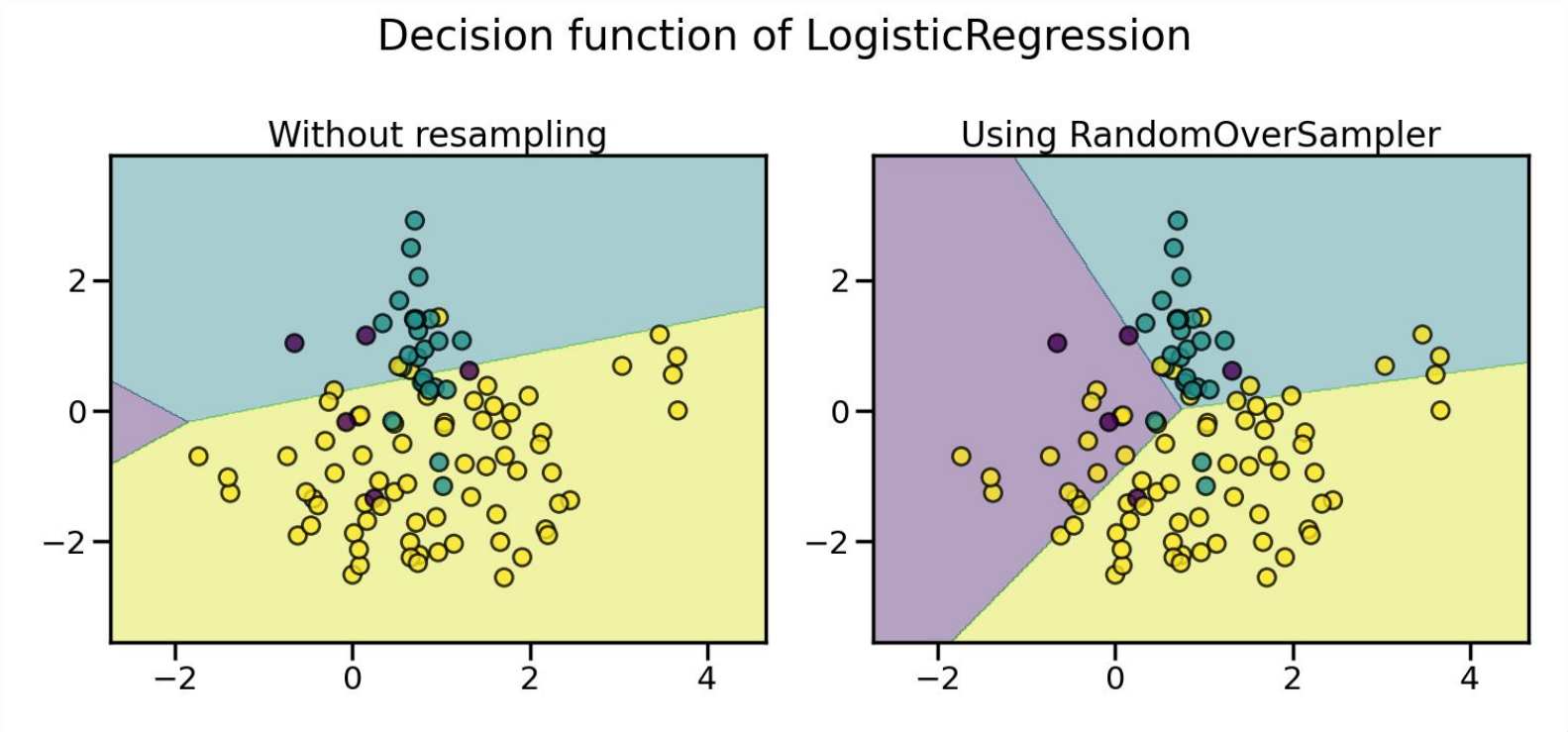
- Common approaches:
  - Random oversampling
  - Smoothed random oversampling
  - SMOTE and variants

- **Main risk**: overfitting duplicated or artificial samples

Random oversampling

- Randomly duplicates samples from the minority class
- Simple and often effective as a baseline
- Does not create new information
  - May cause the model to memorize duplicated samples
- In python: **imblearn**.over_sampling.RandomOverSampler
  - https://imbalanced-learn.org/stable/references/generated/imblearn.over_sampling.RandomOverSampler.html

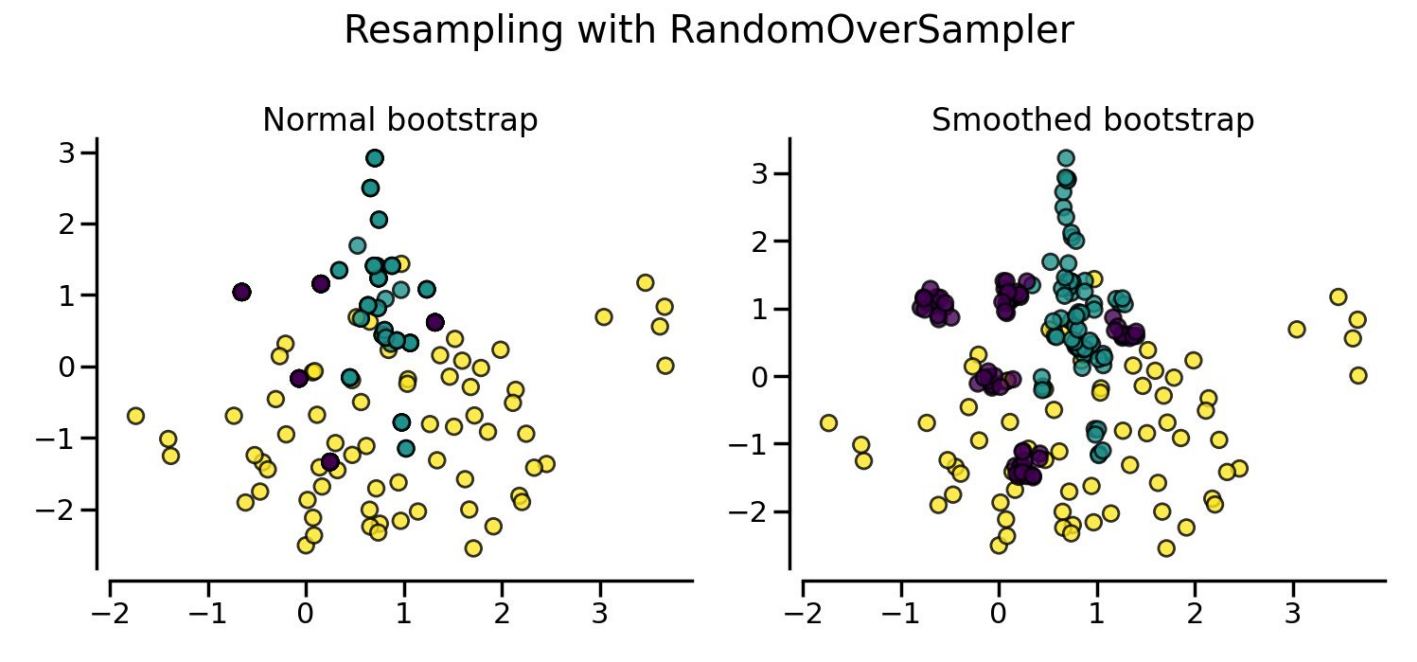
Random oversampling



Smoothed random oversampling

- Adds small perturbations to duplicated samples
  - Reduces exact duplication
- Controlled by the **shrinkage** parameter
- Useful when small variations are meaningful in the feature space

Smoothed random oversampling

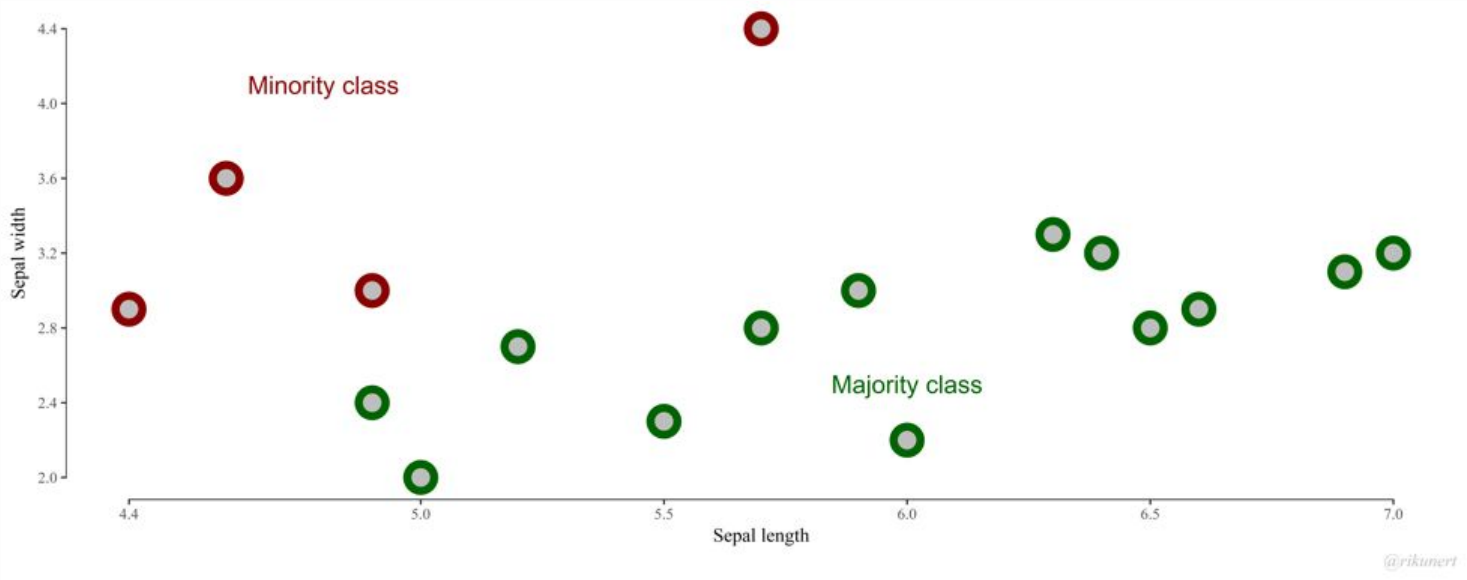


SMOTE

- **S**ynthetic **M**inority **O**versampling **TE**chnique
- Generates **synthetic samples** for the minority class instead of duplicating existing ones
- Works by **interpolating** between a sample and its nearest neighbors from the same class
    - Useful when the minority class has a meaningful continuous feature space

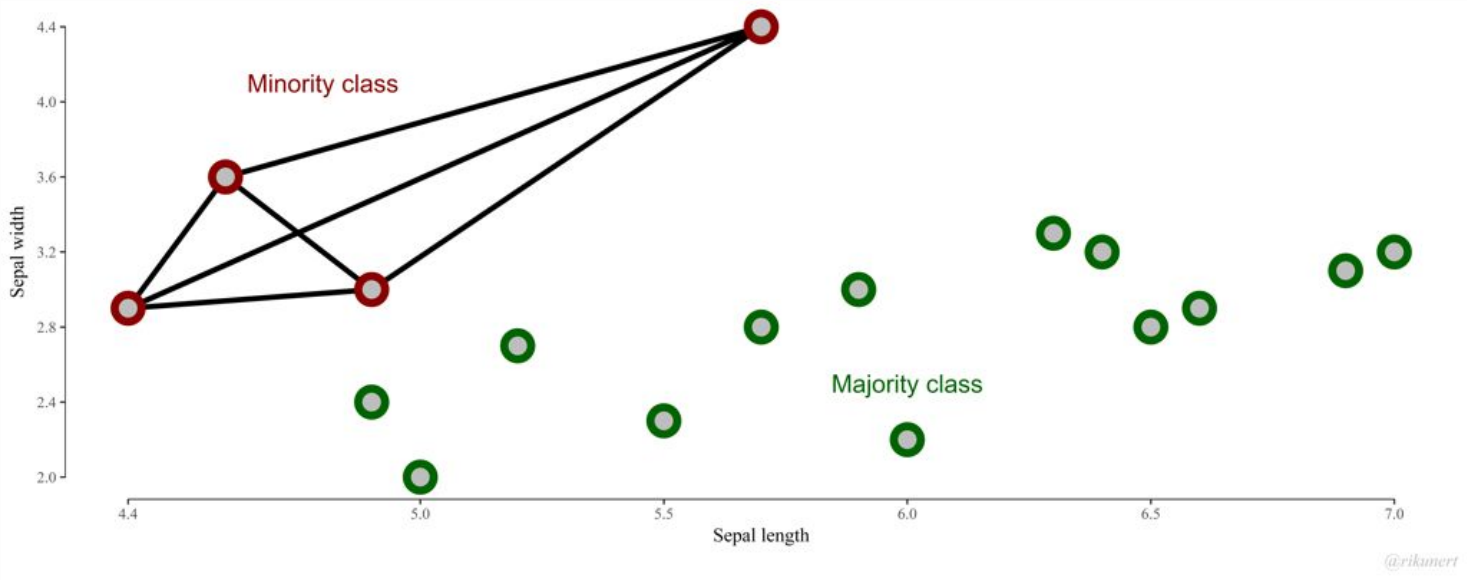
SMOTE – Step 1

Identifying the minority class



SMOTE – Step 2

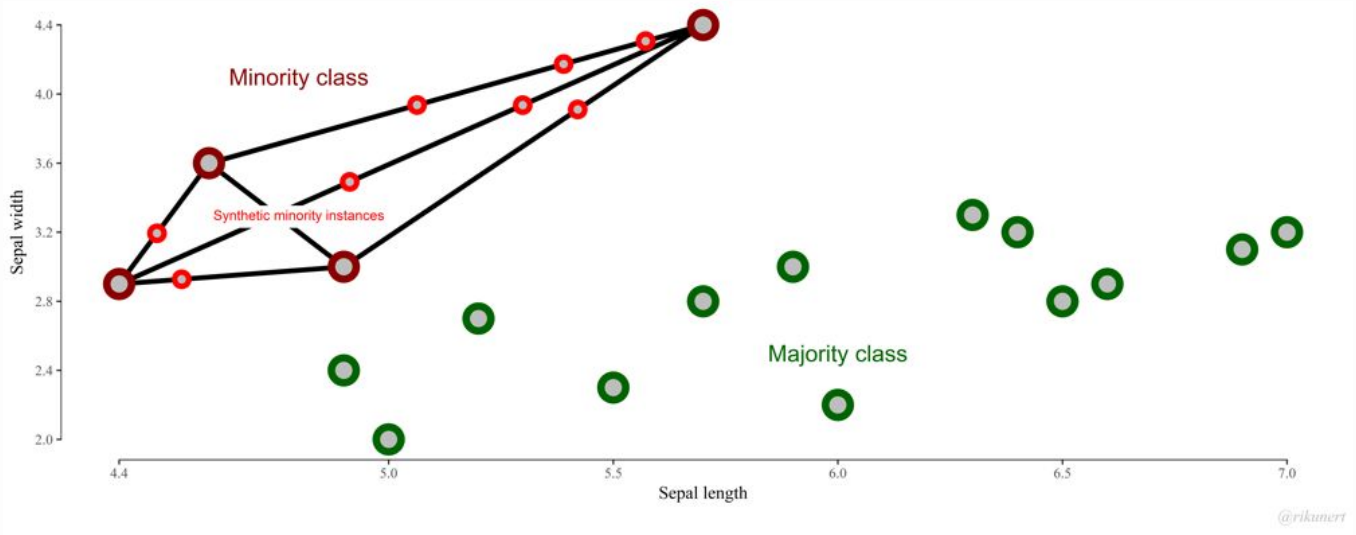
Finding potential regions: connecting the dots



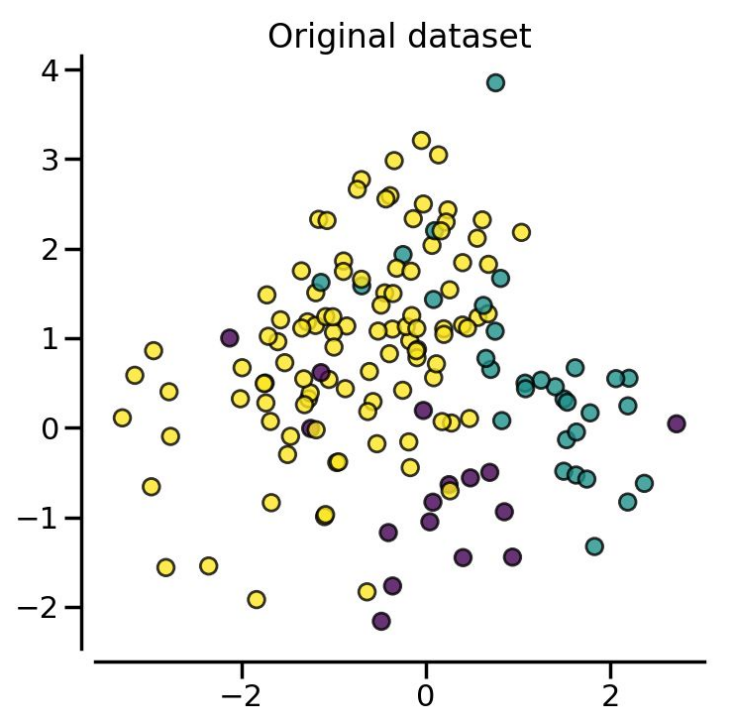
SMOTE – Step 3

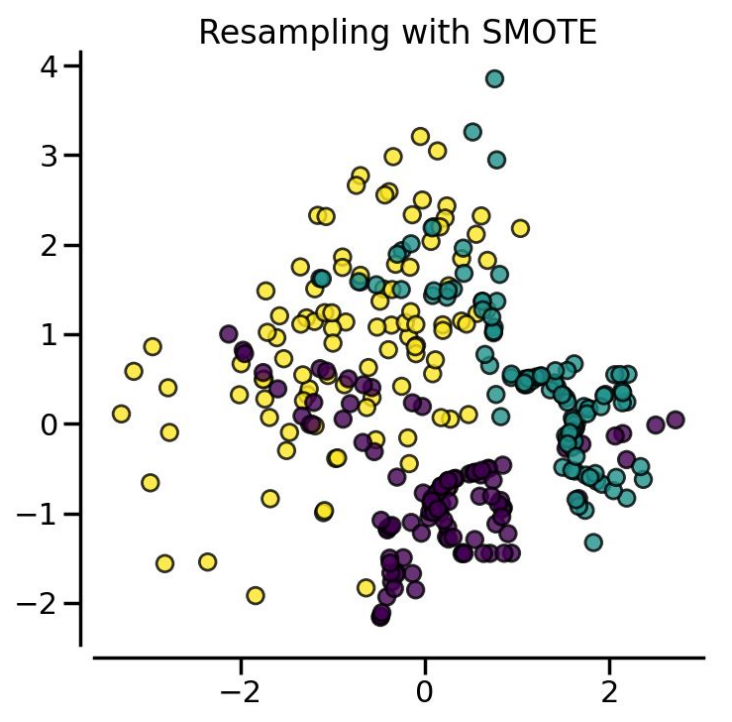
Synthesizing new data points:

1. Generate a synthetic sample by interpolating between a randomly selected instance and one of its **k** nearest neighbors, chosen at random
2. Repeat this process until the desired number of synthetic samples has been generated



SMOTE – Example





SMOTE: Strengths and limitations

- **Strengths:**
  - Reduces exact duplication
  - Can improve minority-class decision boundaries

- **Limitations:**
  - May create unrealistic samples
  - Can amplify noise or outliers
  - Can blur class boundaries when classes overlap

UNDERSAMPLING

Undersampling

- Removes samples from the majority class(es)
- Reduces imbalance and training cost
- **Main risk**: discarding useful information

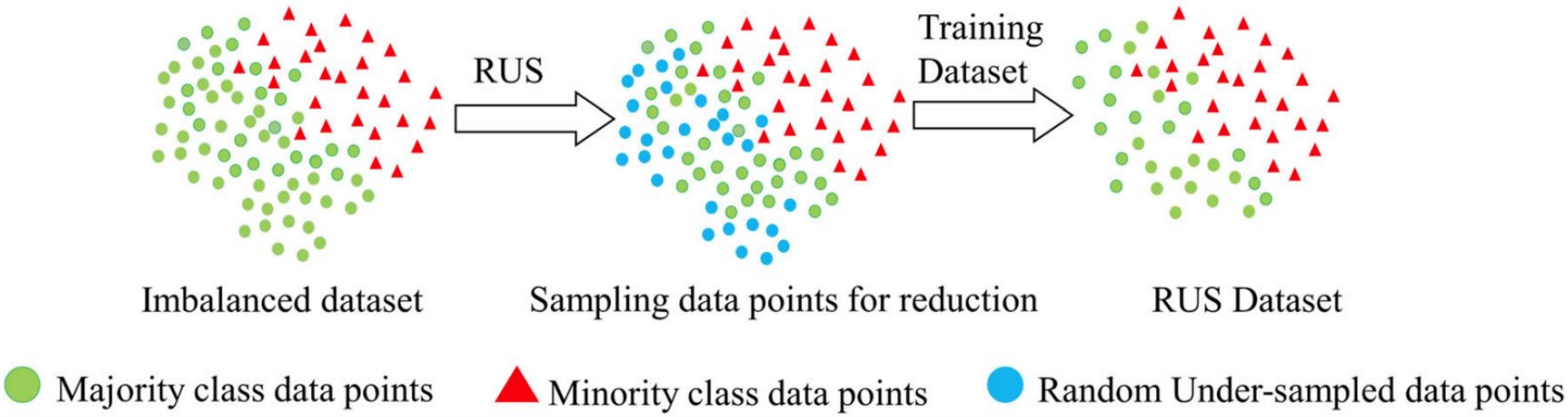
Undersampling

- Removes samples from the majority class(es)
- Reduces imbalance and training cost
- **Main risk**: discarding useful information
  - **Key question**: *which instances should be removed?*

Random undersampling

- Randomly removes majority-class samples
- Simple and fast
- Useful when the majority class is very large
- In python: **imblearn**.under_sampling.RandomUnderSampler
  - https://imbalanced-learn.org/stable/references/generated/imblearn.under_sampling.RandomUnderSampler.html

Random undersampling



NearMiss

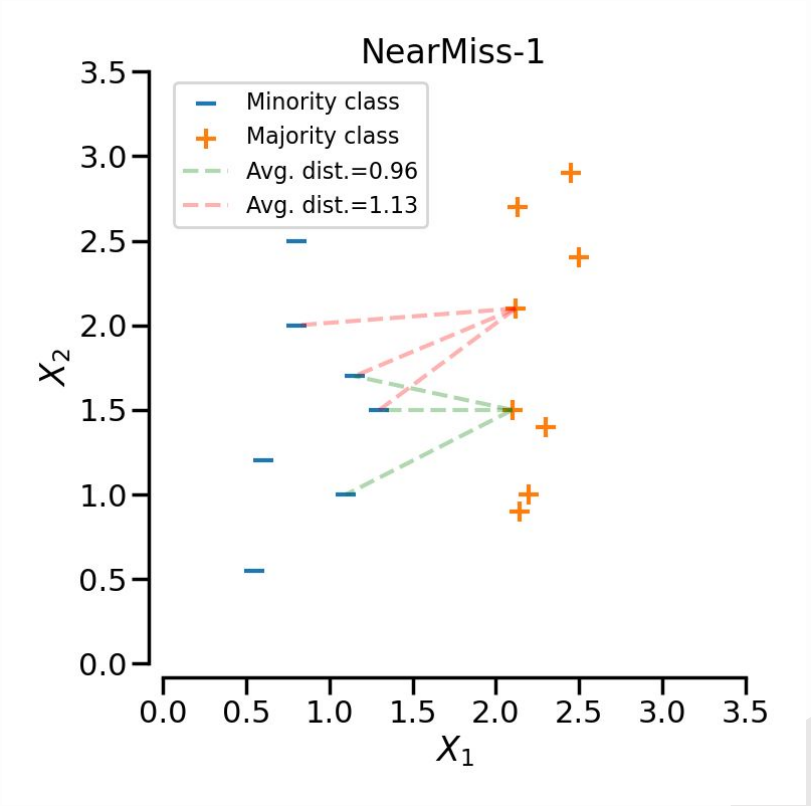
- Selects majority-class samples based on their distance to minority-class samples, rather than randomly removing them

- **Rationale**: aims to retain majority samples that are most informative for distinguishing between classes

- In python: **imblearn**.under_sampling.NearMiss
  - https://imbalanced-learn.org/stable/references/generated/imblearn.under_sampling.NearMiss.html

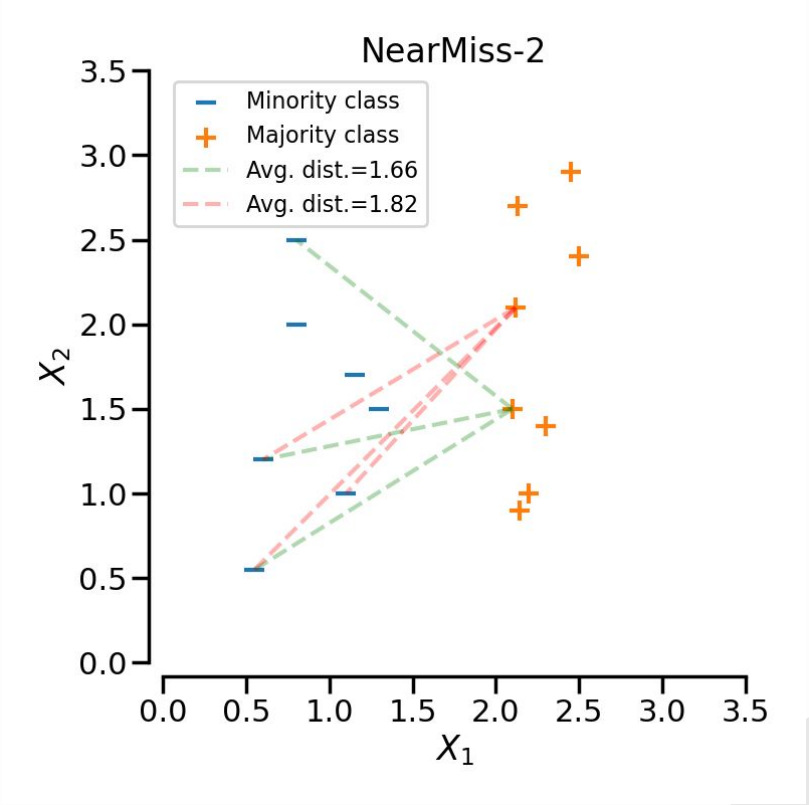
NearMiss-1

- Selects the **majority class** samples for which the average distances to the ***N*** closest samples of the **minority class** are the smallest



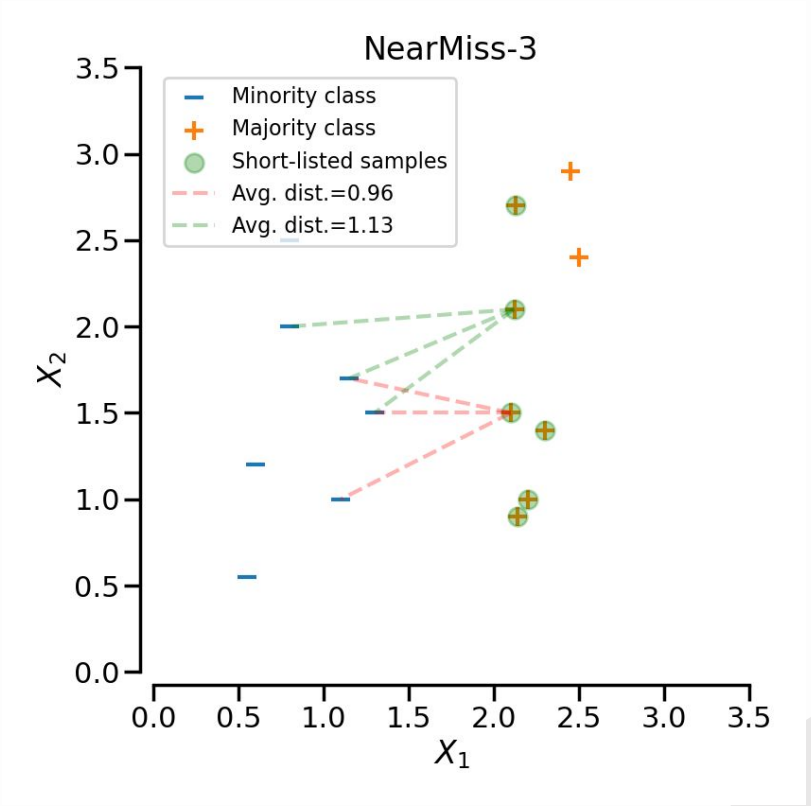
NearMiss-2

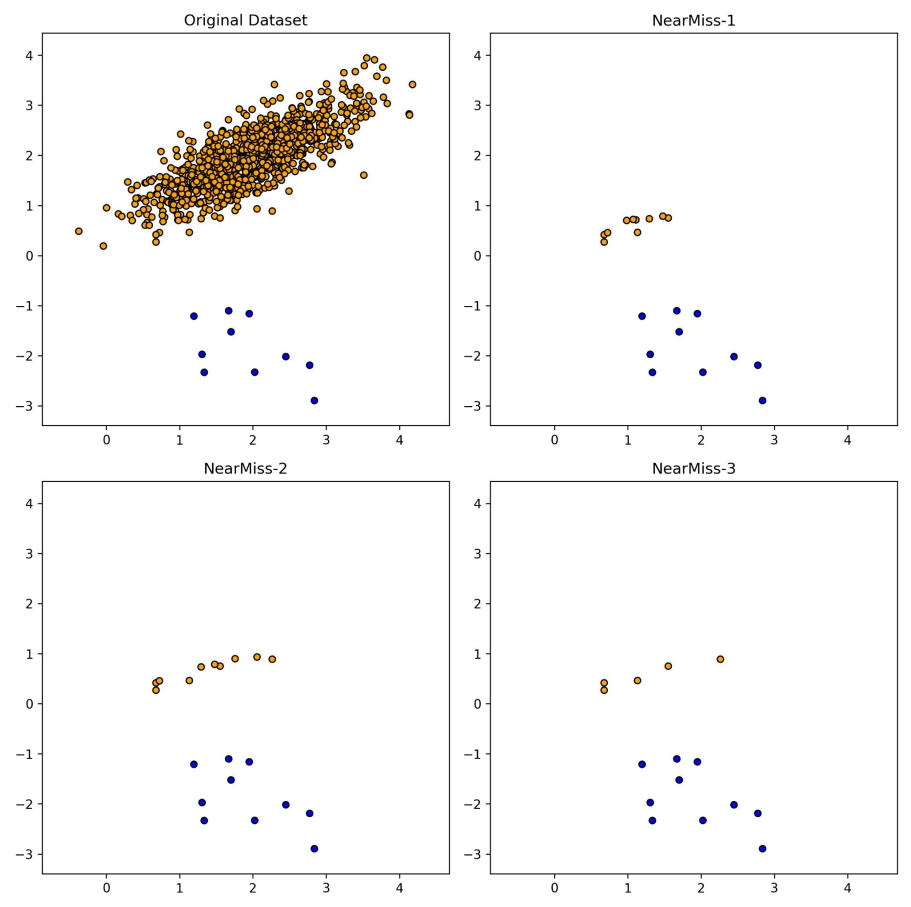
- Selects the **majority class** samples for which the average distances to the ***N*** farthest samples of the **minority class** are the smallest



NearMiss-3

- It is a **two-step algorithm**
- First, for each **minority class** sample, its ***M*** nearest-neighbors are kept
- Then, the **majority class** samples selected are those with the largest average distances to their ***N*** nearest neighbors




Original Dataset
NearMiss-1
NearMiss-2
NearMiss-3

NearMiss Variants – Summary

- **NearMiss-1**: keeps majority samples closest to minority samples
- **NearMiss-2**: considers distance to the farthest minority samples
- **NearMiss-3**: first selects majority neighbors around each minority sample

- **Takeaway**: different variants define “informative majority samples” differently

Choosing a resampling strategy

- Use **random oversampling** as a simple baseline
- Use **SMOTE** when interpolation makes sense
- Use **undersampling** when the majority class is very large

- Always compare strategies using validation data
  - The test set must reflect the original, real-world distribution, including any class imbalance

Practical workflow for imbalanced learning

- Split data into train, validation, and test sets
- Apply resampling only on the training set
- Train the model
- Tune thresholds and hyperparameters on validation data
- Report final performance on the untouched test set

Practice

- Let’s implement all the sampling techniques we’ve discussed using Python and the imblearn library

HYPOTHESIS TESTING

Hypothesis testing

- Used to check whether an observed difference is likely to be meaningful
- Helps avoid conclusions based only on random variation
- Common use case: comparing two or more classifiers
  - Answers: “Is this performance difference statistically significant?”

Null and alternative hypotheses

- **Null hypothesis (H_0)**: there is no meaningful difference
- **Alternative hypothesis (H_1)**: there is a meaningful difference
- Example:
  - H_0: classifier A and classifier B perform similarly
  - H_1: classifier A and classifier B perform differently

Understanding the p-value

- The **p-value** is a number between 0 and 1 that indicates the probability of incorrectly rejecting the null hypothesis (H_0)

- Small p-values suggest that the observed difference is unlikely under H_0

- Common thresholds:
  - **$p > 0.05$:** the observed difference is not statistically significant
  - **$p \leq 0.05$:** the observed difference is significant (95% confidence)
  - **$p \leq 0.01$:** the observed difference is significant (99% confidence)
  - **$p \leq 0.001$:** the observed difference is highly significant (99.9%)

Statistical significance vs. Practical Importance

- A small p-value suggests that the observed difference is unlikely to be due to random variation

- However, this does not mean the difference is practically relevant
  - Example: a model improving from 90.0% to 90.2% accuracy may be statistically significant, but the improvement may be too small to matter

- Practical importance depends on the application, cost, risk, and expected impact

Paired and unpaired tests

- An important consideration when choosing a statistical test is whether the data are **paired** or **unpaired**

- Consider comparing two classifiers using **k-fold cross-validation**:
  - If both classifiers are evaluated using the **exact same folds**, the comparison is **paired** (one-to-one correspondence between samples)

  - If each classifier is evaluated using **different folds**, the comparison is **unpaired** (no direct correspondence between samples)

Parametric and non-parametric tests

- **Parametric tests** assume specific characteristics (often normality) about the population distribution
- **Non-parametric tests** make fewer assumptions — they are distribution-free and suitable for non-Gaussian data

- Best practices:
  - Always **check the data distribution** before selecting a test
  - **Rule of thumb**: with more than **30 samples (e.g., runs/datasets)**, parametric tests are generally safe to use (*Central Limit Theorem*)
  - **When in doubt**, prefer a non-parametric test — they offer greater robustness against violations of distributional assumptions

Main tests

Parametric test	Non-parametric equivalent
Paired t-test	Wilcoxon signed-rank test
Unpaired t-test	Mann-Whitney U test

Choosing a test

- Same folds or same datasets?
  - Use a **paired** test
- Independent results?
  - Use an **unpaired** test
- Approximately normal differences?
  - Consider a **parametric test**
- Non-normal or small sample?
  - Prefer a **non-parametric** test

Practice

- Let’s implement the statistical tests discussed to compare three classifiers

MULTIPLE COMPARISONS

Multiple Comparisons

- What if we need to compare multiple (>2) methods?
- When comparing more than two models, conducting multiple pairwise comparisons increases the risk of **false positives**!
- A statistically sound alternative is to use global tests designed for comparing multiple methods simultaneously

Run	LR	DT	KNN
1	70.12	90.89	81.12
2	75.23	73.18	82.92
3	75.12	75.12	70.89
4	...	...	...

Example with LR, DT, and KNN:

- LR vs. DT
- LR vs. KNN
- DT vs. KNN

Friedman test

- From the ranking matrix,
  the average rank per method is computed: $$\bar{r}_j = \frac{1}{n} \sum_{i=1}^{n} r_{i,j}$$

- The test statistic is computed as: $$Q = \frac{12n}{k(k+1)} \sum_{j=1}^{k} \left(\bar{r}_j - \frac{k+1}{2} \right)^2$$

- And it has a p-value associated
  - If you find statistical difference with Friedman, you need to proceed with a post-hoc test

Two-step statistical testing

- Step 1: use an **omnibus test**
  - Checks whether at least one method differs
  - Example: Friedman test

- Step 2: use a **post-hoc test**
  - Identifies which methods differ
  - Example: Nemenyi test

Friedman Test

- Used for comparing multiple methods on paired results
- Converts raw scores into **ranks**
  - Best method receives rank 1; ties receive average ranks

Run	LR	DT	KNN
1	70.12	90.89	81.12
2	75.23	73.18	82.92
3	75.12	75.12	70.89
4	...	...	...

Rank extraction

Run	LR	DT	KNN
1	3	1	2
2	2	3	1
3	1.5	1.5	3
4	...	...	...

- Tests whether the average ranks differ more than expected by chance

From scores to ranks

Run	LR	DT	KNN
1	70.12	90.89	81.12
2	75.23	73.18	82.92
3	75.12	75.12	70.89
4	...	...	...

Run	LR	DT	KNN
1	3	1	2
2	2	3	1
3	1.5	1.5	3
4	...	...	...

- Organize results as:
  - Rows: datasets, folds, or runs
  - Columns: methods
- Convert each row into ranks
- Compute the **average rank** for each classifier
  - Lower average rank means better overall performance

From scores to ranks

- The Friedman test gives us a **p-value** indicating whether the average ranks differ more than expected by chance
- But this p-value only tells us that **at least one method differs**
- It does not tell us **which methods are significantly different from each other**

Nemenyi Post-hoc Test

- The **Nemenyi test** is used after a significant Friedman test
- It compares pairs of methods based on their **average ranks**
- Two methods are significantly different only if their rank difference is larger than the **Critical Distance**
- This helps identify **which methods actually differ**, not just whether some difference exists

Nemenyi test

- Assuming a confidence level and the number of models, we find its significance level α and the critical value q_α

Critical values for the two-tailed Nemenyi test

# of models	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$q_{0.01}$	2.576	2.913	3.113	3.255	3.364	3.452	3.526	3.590	3.646	3.696	3.741	3.781	3.818	3.853
$q_{0.05}$	1.960	2.344	2.569	2.728	2.850	2.948	3.031	3.102	3.164	3.219	3.268	3.313	3.354	3.391
$q_{0.10}$	1.645	2.052	2.291	2.460	2.589	2.693	2.780	2.855	2.920	2.978	3.030	3.077	3.120	3.159

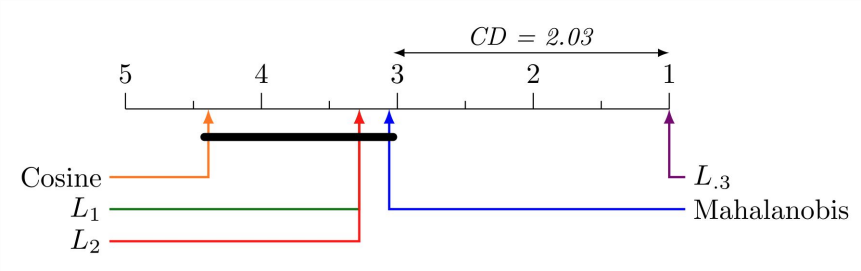
# of models	16	17	18	19	20	21	22	23	24	25	26	27	28	29
$q_{0.01}$	3.884	3.914	3.941	3.967	3.992	4.015	4.037	4.057	4.077	4.096	4.114	4.132	4.148	4.164
$q_{0.05}$	3.426	3.458	3.489	3.517	3.544	3.569	3.593	3.616	3.637	3.658	3.678	3.696	3.714	3.732
$q_{0.10}$	3.196	3.230	3.261	3.291	3.319	3.346	3.371	3.394	3.417	3.439	3.459	3.479	3.498	3.516

# of models	30	31	32	33	34	35	36	37	38	39	40	41	42	43
$q_{0.01}$	4.179	4.194	4.208	4.222	4.236	4.249	4.261	4.273	4.285	4.296	4.307	4.318	4.329	4.339
$q_{0.05}$	3.749	3.765	3.780	3.795	3.810	3.824	3.837	3.850	3.863	3.876	3.888	3.899	3.911	3.922
$q_{0.10}$	3.533	3.550	3.567	3.582	3.597	3.612	3.626	3.640	3.653	3.666	3.679	3.691	3.703	3.714

# of models	44	45	46	47	48	49	50
$q_{0.01}$	4.349	4.359	4.368	4.378	4.387	4.395	4.404
$q_{0.05}$	3.933	3.943	3.954	3.964	3.973	3.983	3.992
$q_{0.10}$	3.726	3.737	3.747	3.758	3.768	3.778	3.788

Nemenyi test

- With these values, we computed the **Critical Distance (CD)**
- CD = $q_\alpha \sqrt{\frac{k(k+1)}{6N}}$, where *N* is the number of methods
- With the CD and the average ranks, we compute critical distance charts:



Note

- The **Friedman test**, followed by Nemenyi post-hoc, is specifically designed for multiple comparisons involving **more than two methods** evaluated on **paired samples**

- For **pairwise comparisons** (e.g., just DT vs KNN), the Friedman test is statistically inappropriate and less powerful than tests designed for two groups, such as:
  - Paired t-test (parametric);
  - Wilcoxon signed-rank test (non-parametric)

Practice

- We will apply a combination of Friedman and Nemenyi post-hoc test to compare multiple classifiers

Take-home messages

- Class imbalance can make accuracy misleading
- Resampling must be applied only to training data
- Evaluation metrics must reflect the goal of the problem
- Statistical tests help avoid overclaiming small differences